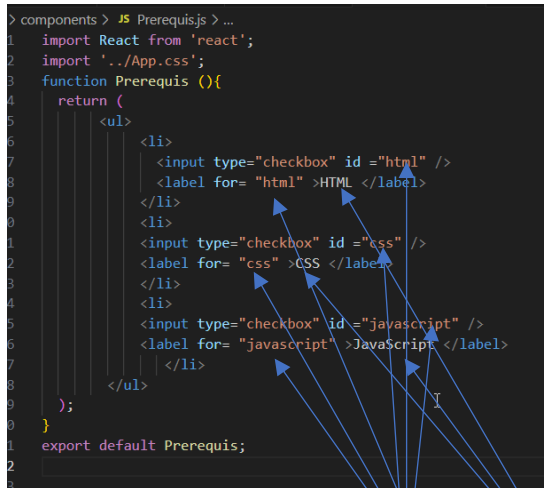


Les props (abréviation de "properties")

En JavaScript, et plus spécifiquement dans le contexte de bibliothèques comme React, les **props** (abréviation de "properties") sont des objets qui permettent de passer des données d'un composant parent à un composant enfant. Les props sont utilisées pour rendre les composants dynamiques et réutilisables.



```

> components > JS Prerequis.js > ...
1 import React from 'react';
2 import '../App.css';
3 function Prerequis () {
4   return (
5     <ul>
6       <li>
7         <input type="checkbox" id="html" />
8         <label for="html">HTML </label>
9       </li>
10      <li>
11        <input type="checkbox" id="css" />
12        <label for="css">CSS </label>
13      </li>
14      <li>
15        <input type="checkbox" id="javascript" />
16        <label for="javascript">JavaScript </label>
17      </li>
18    </ul>
19  );
20 }
21 export default Prerequis;
22

```

Comme vous remarquez sur la figure ci-dessus, on a trois `<li>` qui se répètent, et il y a trois variables qui changent le `id`, `for` et le nom du langage. On peut penser à créer un seul composant qui s'appelle `ElementPrerequis` et on l'appelle à chaque fois qu'on veut afficher un prérequis.

On va créer un fichier `ElementPrerequis.js`

```

import React from 'react';
import '../App.css';
function ElementPrerequis (props) {
  return (
    <li>
      <input type={props.type} id={props.id} />
      <label for={props.id}>{props.nom} </label>
    </li>
  );
}
export default ElementPrerequis;

```

Et on va modifier `Prerequis.js`

```

import React from 'react';
import '../App.css';
import ElementPrerequis from './ElementPrerequis';
function Prerequis () {
  return (
    <ul>
      <ElementPrerequis id="html" nom="HTML" />

```

```

        <ElementPrerequis id="css" nom="CSS" />
        <ElementPrerequis id="javascript" nom="JavaScript" />

    </ul>

);
}
export default Prerequis;

```

Dans l'exemple précédent on a utilisé :

Composants fonctionnels : Dans un composant fonctionnel, les props sont passées comme argument à la fonction. Vous pouvez les utiliser directement dans le corps de la fonction.

Il y a aussi ce qu'on appelle :

Composants de classe : Dans un composant de classe, les props sont accessibles via `this.props`. React passe automatiquement les props à chaque instance du composant, ce qui permet de les utiliser directement dans les méthodes du composant, y compris `render`.

Dans les composants de classe, vous n'avez pas besoin de passer explicitement `props` à la méthode `render` car React gère cela en arrière-plan. C'est pourquoi vous pouvez accéder aux props via `this.props` directement.

```

import { Component } from 'react';
class ElementPrerequis extends Component {
  render () {
    return (

      <li>
        <input type="checkbox" id= {this.props.technologieId}/>
        <label htmlFor={this.props.technologieId}>
{this.props.technologieName} </label>
      </li>
    )
  }
}
export default ElementPrerequis;

```

```

import { Component } from 'react';
import ElementPrerequis from './ElementPrerequis';
class Prerequis extends Component {
  render () {
    return (
      <ul>
        <ElementPrerequis technologieId="html" technologieName="HTML"/>
        <ElementPrerequis technologieId="css" technologieName="CSS"/>
        <ElementPrerequis technologieId="javaScript" technologieName="JAVASCRIPT"/>
      </ul>
    )
  }
}

```

```

    )
  }
}
export default Prerequis;

```

Ajout d'autres propriétés

On veut afficher les images des prérequis.

React est une bibliothèque JavaScript populaire utilisée pour construire des interfaces utilisateur (UI). Développée par Facebook, elle permet de créer des applications web interactives et dynamiques de manière efficace. Avant de commencer avec React, il est utile de maîtriser certains concepts et technologies de base. Voici une liste des éléments clés à connaître :

-  ☐ HTML
-  ☐ CSS
-  ☐ JAVASCRIPT

On va créer un répertoire images sous le répertoire public

On met dedans les trois images correspondantes au outils (avec drag and drop) html, css et javascript

Les images sont dans le répertoire C:\Users\pc\Dropbox\2025-cours
javascript\Javascript\images

```

import React from 'react';
import '../App.css';
import ElementPrerequis from './ElementPrerequis';
function Prerequis (){
  return (
    <ul>
      <ElementPrerequis id="html" nom="HTML" image="\images\html.png"/>
      <ElementPrerequis id="css" nom="CSS" image="\images\css.png"/>
      <ElementPrerequis id="javascript" nom="JavaScript"
image="\images\javascript.png"/>
    </ul>
  );
}
export default Prerequis;

```

```

import React from 'react';
import '../App.css';
function ElementPrerequis (props){
  return (
    <li>

```

```

    <img src={props.image} alt={`image de ${props.nom}`} width="20"/>
    <input type={"checkbox"} id={props.id} />
    <label htmlFor={props.id} >{props.nom} </label>
  </li>

);
}
export default ElementPrerequis;

```

On peut écrire avant `return console.log(props)` pour voir ce qu'elle contient.

On peut passer les paramètres autrement, n'oublier les `{}` pour image, nom et id :

```

import React from 'react';
import '../App.css';
function ElementPrerequis ({image,id,nom}){
  return (
    <li>
      <img src={image} alt={nom} width="20"/>
      <input type={"checkbox"} id={id} />
      <label htmlFor={id} >{nom} </label>
    </li>
  );
}
export default ElementPrerequis;

```

Quand vous utilisez un composant React comme `<ElementPrerequis image="mon_image.jpg" id={123} nom="Mon Élément" />`, React regroupe toutes ces "propriétés" (image, id, nom) dans un unique objet, appelé props.

Avec Déstructuration (Notre Cas) : La syntaxe `{image, id, nom}` dans les paramètres de la fonction dit à JavaScript : "Je sais que je vais recevoir un objet en premier argument. Au lieu de me donner l'objet entier, s'il te plaît, extrais directement les propriétés nommées image, id, et nom de cet objet et donne-les-moi comme des variables locales distinctes."

Avec des composants de type classe :

```

import { Component } from 'react';
import ElementPrerequis from './ElementPrerequis';
class Prerequis extends Component {
  render () {
    return (
      <ul>
        <ElementPrerequis technologieId="html" technologieName="HTML"
image="/images/html.png"/>
        <ElementPrerequis technologieId="css" technologieName="CSS"
image="/images/css.png"/>
        <ElementPrerequis technologieId="javaScript"
technologieName="JAVASCRIPT" image="/images/javascript.png"/>
      </ul>
    );
  }
}

```

```

    )
  }
}
export default Prerequis;

```

```

import { Component } from 'react';
class ElementPrerequis extends Component {
  render () {
    return (
      <li> <img src={this.props.image} alt="" width="20px"/>
        <input type="checkbox" name= {this.props.technologieId} id=
{this.props.technologieId}/>
        <label htmlFor ={this.props.technologieId}>
{this.props.technologieName} </label>
      </li>
    )
  }
}
export default ElementPrerequis;

```

Par contre si on met les images dans un répertoire autre que public, ça ne va pas fonctionner. Imaginons qu'on met les images dans un répertoire images sous le répertoire 'src'

Les fichiers dans `src/` sont traités comme des **modules JavaScript**.

Requiert un `import` pour être reconnu par le bundler (Webpack) :

Explication de **bundler (Webpack)** :

1. Vos Fichiers de Code :

- Quand tu écris une application React, tu ne crées pas un seul énorme fichier. Tu crées plein de petits fichiers JavaScript (.js ou .jsx) pour chaque composant (un bouton, un formulaire, une page...).
- Tu utilises aussi du CSS pour le style, peut-être des images, des polices d'écriture, etc.
- Dans tes fichiers JavaScript, tu utilises des `import` pour utiliser un composant dans un autre, et `export` pour rendre un composant utilisable. Tu utilises aussi du JSX (le truc qui ressemble à du HTML dans ton JavaScript), et peut-être du JavaScript très récent.

2. Le Problème (Le Navigateur est un peu Bête):

- Le navigateur web (Chrome, Firefox...) ne comprend pas directement tout ça nativement :
 - Il ne comprend pas les `import/export` comme ça.

- Il ne comprend pas le JSX.
- Il ne comprend pas forcément les toutes dernières fonctionnalités JavaScript.
- Charger 100 petits fichiers est beaucoup plus lent que charger 1 ou 2 gros fichiers bien organisés.

3. Webpack (Le Chef de Chantier Intelligent):

- **Webpack est un "bundler" (un "regroupeur" ou "assembleur").** Son travail principal est de prendre tous tes fichiers et de les transformer et les assembler intelligemment en quelques fichiers que le navigateur comprendra parfaitement et pourra charger rapidement.

Ce que fait Webpack (en très simple) :

- **Il Regarde:** Il commence par ton fichier principal (souvent index.js) et regarde quels autres fichiers tu import. Puis il regarde ce que ces fichiers importent, et ainsi de suite. Il crée une carte de toutes les dépendances (qui a besoin de quoi).
- **Il Transforme:** Pendant qu'il regarde tes fichiers, il utilise des outils (appelés "loaders") pour les transformer :
 - Il transforme ton JSX en JavaScript normal.
 - Il transforme ton JavaScript moderne en une version que même les navigateurs plus anciens comprennent (avec un outil comme Babel).
 - Il peut comprendre tes import de CSS et les regrouper en un seul fichier CSS.
 - Il peut même optimiser tes images.
- **Il Assemble (Bundle):** Finalement, il prend tout ce code JavaScript transformé et le regroupe en un seul (ou quelques) gros fichier(s) JavaScript. Pareil pour le CSS. Ces fichiers finaux sont appelés des "bundles".
- **Il Optimise:** Il peut aussi faire des choses intelligentes comme supprimer le code inutile ou le rendre plus petit (minification) pour que ça charge encore plus vite.

htmlImage: C'est le nom de la variable que vous utilisez pour référencer l'image importée dans votre code actuel. Vous pouvez choisir n'importe quel nom de variable valide ici (par exemple, myHtmlImage, logoHtml, etc.). Le nom htmlImage est une convention logique qui suggère qu'elle représente une image HTML.

```
import { Component } from 'react';
import ElementPrerequis from './ElementPrerequis';
import htmlImage from '../images/html.png';
import cssImage from '../images/css.png';
import javascriptImage from '../images/javascript.png';

class Prerequis extends Component {
  render () {
    const imagehtml = htmlImage;
    const imagecss = cssImage;
    const imagejavascript = javascriptImage;

    return (
      <ul>
        <ElementPrerequis technologieId="html" technologieName="HTML"
image={imagehtml}/>
      </ul>
    );
  }
}
```

```
        <ElementPrerequis technologieId="css" technologieName="CSS"
image={imagecss}/>
        <ElementPrerequis technologieId="javaScript"
technologieName="JAVASCRIPT" image={imagejavascript}/>
    </ul>
)
}
}
export default Prerequis;
```

Le State, en termes simples, est la "mémoire" d'un composant React. C'est un objet JavaScript qui contient des données qui peuvent changer au fil du temps.

Les changements de State déclenchent une réactualisation (re-render) du composant, ce qui permet à l'interface utilisateur de se mettre à jour en fonction des nouvelles données.

les Hooks sont le *moyen* de gérer le State dans les composants fonctionnels React.

les hooks

Les hooks en JavaScript sont une fonctionnalité introduite en 2019 avec React 16.8. Ils permettent d'ajouter des fonctionnalités aux composants fonctionnels sans utiliser de classes.

Exemples simples et concrets

useState : Gérer un compteur

Le State

Pensez à un compteur :

- Il doit se souvenir du nombre actuel.
- Quand tu cliques sur "+", ce nombre doit changer.
- Le composant doit afficher le *nouveau* nombre.
- Le "state" est parfait pour garder en mémoire ce nombre qui change.

Quand le "state" d'un composant change, React **redessine automatiquement** ce composant à l'écran pour montrer la nouvelle information.

Les Hooks

- Autrefois, seuls les "Class Components" (une façon plus ancienne et plus complexe d'écrire des composants) pouvaient avoir une mémoire (state) et faire d'autres choses avancées. Les "Function Components" (la façon plus simple) étaient juste bons à afficher des choses, sans mémoire.
- **La Solution : Les Hooks !** Les Hooks sont des **fonctions spéciales** (elles commencent toujours par use...) que React vous donne pour que tes **composants fonctions** (les simples) puissent utiliser des super-pouvoirs de React, comme :

- Avoir une mémoire (le state).
- Réagir à des événements extérieurs (comme quand le composant apparaît ou disparaît).
- Accéder à des informations générales de l'application.

Le Hook le Plus Important pour le State : useState

- **useState** : C'est l'**outil (Hook)** principal pour donner de la **mémoire (State)** à un composant simple. Il te donne une variable pour lire la valeur et une fonction spéciale pour la modifier (ce qui déclenche le redessin).

Avec **useState** (nouveau React) :

On va créer un composant Hooks.js dans le répertoire components

```
import React, { useState } from 'react';
function Hooks () {
  const [count, setCount]=useState(0);
  function handelClick(){
    setCount(count+1)
  }
  // const handelClick =()=>{
  //   setCount(count+1)
  // }

  return (
    <div>
      vous avez cliquez {count}

      <br/>

      <button onClick={handelClick}> Incrementer par 1 </button>
    </div>
  );
}
export default Hooks;
```

- **Explication :**
 - useState(0) : Crée une variable d'état appelée compteur et l'initialise à 0. Renvoie un tableau :
 - count : La valeur actuelle de l'état (au début, c'est 0).
 - setCount : Une fonction pour mettre à jour la valeur de l'état.
 - setCount(count + 1) : Met à jour la valeur de compteur en ajoutant 1 à sa valeur actuelle. Lorsque tu appelles setCount, React met à jour le composant et l'affiche à nouveau avec la nouvelle valeur.

Il faut remarquer que chaque fois que la valeur de count change, React render (affiche) le nouveau contenu de la page automatiquement.

Et on va l'appeler dans index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import Hooks from './components/Hooks';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <Hooks/>
  // <App />
);
```

Imaginons ce cas :

```
function Hooks() {
  const [compteur, setCompteur]=useState(0)
  function handleClick(){
    let compteur=compteur+1
    setCompteur(compteur)
  }
  return (
    <div>
```

Ici on essaye de changer une variable d'état (compteur) créée avec useState sans utiliser la fonction fournie par useState.

La Règle Fondamentale :

Pour changer une variable d'état créée avec useState, tu dois **TOUJOURS** et **UNIQUEMENT** utiliser la fonction fournie par useState (ici, setCompteur) et lui donner la **nouvelle valeur** que tu souhaites.

On ne doit **JAMAIS** modifier directement une variable d'état (comme compteur venant de useState).

Soit on fait

```
function Hooks() {
  const [compteur, setCompteur]=useState(0)
  function handleClick(){
    let nouveau=compteur+1
    setCompteur(nouveau)
  }
}
```

Soit encore mieux :

```
function Hooks() {
  const [compteur, setCompteur]=useState(0)
  function handleClick(){
    setCompteur(compteur+1)
  }
}
```

Maintenant on veut avoir la valeur de départ et le pas d'incrémentation comme variable qui proviennent de index.js

Solution

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import Hooks from './components/Hooks';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <Hooks depart="2" step="4"/>
  // <App />
);
```

```
import React, { useState } from 'react';
import './App.css';

function Hooks ({depart,step}) {
  const [count,setCount]=useState(parseInt(depart));
  function handelClick(){
    setCount(count+parseInt(step))
  }
  return (
    <div>
      vous avez cliquez {count}
      <br/>
      <button onClick={handelClick}> Incrémenter par {step} </button>
    </div>
  );
}
export default Hooks;
```

On peut utiliser aussi les props :

```
import React, { useState } from 'react';
function Hooks (props) {
  const [count,setCount]=useState(props.depart)
  function handelClick(){
    setCount(count+props.pas)
  }
  function handelReset(){
    setCount(0)
  }
}
```

```

    return (
      <>
        vous avez cliquer {count} fois
        <br/>
        <button onClick={handelClick}> incrementer par
{props.pas}</button>
        <button onClick={handelReset}> Reintialiser</button>
      </>
    );
  }
export default Hooks;

```

Maintenant on va créer un formulaire avec deux champs nom et âge et on va afficher les valeurs saisies dans la console.

Solution

On va créer un composant Form.js

```

import React, { useState } from 'react';
import '../App.css';

function Form () {
  const [nom,setNom]=useState(null);
  const [age,setAge]=useState(null);
  function handleChange(e){
    let leNom=document.getElementById("nom").value
    setNom(leNom)
    let lAge=document.getElementById("age").value
    setAge(lAge)
  }
  function hundelClick(e){
    e.preventDefault()
    console.log(nom + " " + age)
  }
  return (
    <div>
      Nom : <input type= "text" id="nom" onChange={handleChange} />
      <br/>
      Age : <input type= "number" id="age" onChange={handleChange} />
      <br/>
      <input type = "submit" onClick={hundelClick}/>
    </div>
  );
}

```

```
export default Form;
```

L'argument `e` dans la fonction `handleClick` représente l'objet *Event* qui est automatiquement passé à la fonction lorsqu'un événement (comme un clic) se produit.

e ou event est une convention : Bien que vous puissiez utiliser n'importe quel nom d'argument, `e` ou `event` sont les noms les plus couramment utilisés par les développeurs pour représenter l'objet *Event*.

Quand le navigateur appelle ta fonction `handleSubmit` *parce que* l'utilisateur a cliqué (ou soumis un formulaire, etc.), il ne l'appelle pas les mains vides. Il lui donne automatiquement un **cadeau spécial**, une sorte de **rapport détaillé** sur ce qui vient de se passer. Ce cadeau, c'est l'objet `e` ou *Event*.

Cet objet contient **plein d'informations** sur l'événement qui vient de se produire :

- Quel type d'événement c'était (un clic ? une soumission de formulaire ? une touche de clavier enfoncée ?)
- Quel élément HTML a déclenché l'événement (sur quel bouton a-t-on cliqué ? `e.target`)
- Où était la souris au moment du clic ? (`e.clientX`, `e.clientY`)
- Et plein d'autres détails...

L'objet `e` ou *Event* contient des informations sur l'événement qui s'est produit. Il vous donne accès à des propriétés et des méthodes utiles pour gérer l'événement de manière appropriée. Par exemple : **`e.preventDefault()`** : Dans cet exemple, on utilise `e.preventDefault()`. Cette méthode est très courante et importante. Elle empêche le comportement par défaut de l'élément HTML qui a déclenché l'événement. Par exemple, notre `handleClick` est lié à un formulaire `<form>`, cela empêcherait la soumission du formulaire et le rechargement de la page.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import Hooks from './components/Hooks';
import Form from './components/Form';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <Form />
  //<Hooks depart="2" step="4"/>
  // <App />
);
```